

Two Pointers in Python — DSA Guide

A detailed pattern-first explanation + 7 LeetCode problems with Python solutions

1. Prerequisites: What you should know about arrays

Before learning Two Pointers, be comfortable with Python lists: indexing, `len()`, loops, updating values, `append/pop`, slicing, and sorting. You should also understand basic time complexity: $O(1)$, $O(n)$, and $O(n^2)$.

```
arr = [2, 4, 6, 8]

arr[0]      # first element
arr[-1]     # last element
len(arr)    # number of elements

for i in range(len(arr)):
    print(arr[i])

arr.sort()  # sorts in place
```

2. What is the Two Pointers technique?

Two Pointers is a problem-solving pattern where you keep track of two positions (usually indices) in an array or string. Instead of repeatedly scanning the same elements with nested loops, you move the pointers according to the problem's rules.

Core idea: use two indices to reduce unnecessary work.

```
left = 0
right = len(arr) - 1

while left < right:
    # use arr[left] and arr[right]

    left += 1
    right -= 1
```

3. Pattern 1 — Pointers from opposite ends

This is the best first Two Pointers pattern to learn. One pointer starts at the beginning and the other at the end. They move toward each other.

```
[ 1  2  3  4  5  6 ]
  ↑           ↑
 left       right
```

After one step:

```
[ 1  2  3  4  5  6 ]
      ↑       ↑
    left    right
```

Typical uses: palindrome checks, reversing, pair-sum in sorted arrays, and problems where the two ends give useful information.

4. The pointer-movement rule

The hardest part is usually not writing `left` and `right`. It is deciding **which pointer should move**. A pointer should move when you have logically ruled out the current position.

For example, in Two Sum II with a sorted array: if `arr[left] + arr[right]` is too small, moving right leftward would only make the sum smaller, so move left forward. If the sum is too large, move right backward.

```
if arr[left] + arr[right] == target:
    return [left, right]
elif arr[left] + arr[right] < target:
    left += 1
else:
    right -= 1
```

5. Complexity intuition

If each pointer moves only forward, only backward, or toward each other, each element is usually processed a constant number of times. That gives $O(n)$ time and $O(1)$ extra space for many Two Pointer solutions.

Important: Two Pointers does not automatically mean $O(n)$. Sorting first may add $O(n \log n)$, and some problems such as 3Sum or Trapping Rain Water have additional structure.

LeetCode Practice — Questions + Python Answers

LC 1 — Two Sum

How it works: Classic Two Sum is usually solved with a hash map, not literal Two Pointers, because the array is not guaranteed to be sorted. The hash map stores previously seen values and their indices.

```
def twoSum(nums, target):
    seen = {}

    for i, x in enumerate(nums):
        need = target - x

        if need in seen:
            return [seen[need], i]

        seen[x] = i

    return []
```

Complexity: $O(n)$ time, $O(n)$ extra space.

Example: Example: `nums = [2, 7, 11, 15]`, `target = 9` → `[0, 1]`.

LC 125 — Valid Palindrome

How it works: Use one pointer at each end. Skip characters that are not letters or digits, compare the remaining characters case-insensitively, then move both pointers inward.

```
def isPalindrome(s):
    left = 0
    right = len(s) - 1

    while left < right:
        while left < right and not s[left].isalnum():
            left += 1

        while left < right and not s[right].isalnum():
            right -= 1

        if s[left].lower() != s[right].lower():
            return False

        left += 1
        right -= 1

    return True
```

Complexity: $O(n)$ time, $O(1)$ extra space (ignoring Python's string representation details).

Example: Example: "A man, a plan, a canal: Panama" → True.

LC 167 — Two Sum II

How it works: This is the textbook opposite-end Two Pointers problem. The array is sorted. A sum that is too small means the left value must increase; a sum that is too large means the right value must decrease.

```
def twoSum(numbers, target):
    left = 0
    right = len(numbers) - 1

    while left < right:
        total = numbers[left] + numbers[right]
```

```

        if total == target:
            return [left + 1, right + 1]
        elif total < target:
            left += 1
        else:
            right -= 1

    return []

```

Complexity: $O(n)$ time, $O(1)$ extra space.

Example: Example: [2, 7, 11, 15], target = 9 $\rightarrow$ [1, 2].

LC 344 — Reverse String

How it works: Swap the first and last characters, then move both pointers inward. Repeat until they meet.

```

def reverseString(s):
    left = 0
    right = len(s) - 1

    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1

```

Complexity: $O(n)$ time, $O(1)$ extra space.

Example: Example: ['h','e','l','l','o'] $\rightarrow$ ['o','l','l','e','h'].

LC 11 — Container With Most Water

How it works: The area is width $\times$ the shorter wall. Start with the widest container. To potentially improve the area, move the pointer at the shorter wall: moving the taller wall would reduce width while keeping the limiting height unchanged.

```

def maxArea(height):
    left = 0
    right = len(height) - 1
    best = 0

    while left < right:
        width = right - left
        current = min(height[left], height[right]) * width
        best = max(best, current)

        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return best

```

Complexity: $O(n)$ time, $O(1)$ extra space.

Example: Example: [1,8,6,2,5,4,8,3,7] $\rightarrow$ 49.

LC 15 — 3Sum

How it works: Sort first. Fix one number `nums[i]`, then solve a Two Sum II-style problem on the remaining suffix using left and right pointers. Sorting also lets us skip duplicate values.

```

def threeSum(nums):
    nums.sort()
    result = []

    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:

```

```

        continue

    left = i + 1
    right = len(nums) - 1

    while left < right:
        total = nums[i] + nums[left] + nums[right]

        if total == 0:
            result.append([nums[i], nums[left], nums[right]])

            left += 1
            right -= 1

            while left < right and nums[left] == nums[left - 1]:
                left += 1
            while left < right and nums[right] == nums[right + 1]:
                right -= 1

        elif total < 0:
            left += 1
        else:
            right -= 1

    return result

```

Complexity: $O(n^2)$ time after sorting, $O(1)$ auxiliary space apart from the returned result (sorting space can depend on the implementation).

Example: Example: $[-1,0,1,2,-1,-4] \rightarrow [[-1,-1,2],[-1,0,1]]$.

LC 42 — Trapping Rain Water

How it works: Water above a position depends on the smaller of the tallest wall on its left and the tallest wall on its right. With two pointers, when $\text{height}[\text{left}] \leq \text{height}[\text{right}]$, the left side is the limiting side, so left can be processed using `left_max`. Otherwise process the right side using `right_max`.

```

def trap(height):
    left = 0
    right = len(height) - 1
    left_max = 0
    right_max = 0
    water = 0

    while left < right:
        if height[left] <= height[right]:
            if height[left] >= left_max:
                left_max = height[left]
            else:
                water += left_max - height[left]
            left += 1
        else:
            if height[right] >= right_max:
                right_max = height[right]
            else:
                water += right_max - height[right]
            right -= 1

    return water

```

Complexity: $O(n)$ time, $O(1)$ extra space.

Example: Example: $[0,1,0,2,1,0,1,3,2,1,2,1] \rightarrow 6$.

6. How to recognize a Two Pointers problem

Signal in the problem	What to think
Array/string has two useful ends	Try left = 0 and right = n - 1
Array is sorted + pair/target	Try opposite-end pointers
Need to compare mirrored positions	Try moving inward
Need to reverse in-place	Swap left/right
Need to remove duplicates or compact values	Consider slow/fast pointers
Need to scan a range while maintaining a condition	This may be Sliding Window instead

7. Recommended learning order

Start with LC 344 and LC 125 to understand the mechanics. Then do LC 167 to learn how sorting creates a pointer-movement rule. Next study LC 11. After that, LC 15 combines an outer loop with Two Pointers, and LC 42 teaches a more advanced invariant-based use of two pointers. Keep LC 1 alongside these as a reminder that 'two-sum' does not always mean Two Pointers.

8. A reusable checklist

1. Is the input sorted, or can I sort it?
2. Can I represent the problem with two positions?
3. What does each pointer mean?
4. When is the current pair/position impossible?
5. Which pointer can I safely move?
6. What invariant remains true after the move?
7. When do the pointers stop?
8. What is the time and space complexity?

Key takeaway

Don't memorize pointer movements. Learn the reason behind each movement. In good Two Pointer solutions, every pointer move eliminates a set of possibilities. That is the core idea that turns brute force into an efficient solution.